

Day 88: Redis Basics

1. Day Number

Day 88

2. Topic Name

Key-Value Storage and Caching with Redis

Today you will learn about **Redis**, a fast data store that can save information using simple **keys and values**.

A basic Redis idea looks like:

key	value
product:101:price	1200

You can think of it somewhat like a Python dictionary:

```
{  
    "product:101:price": 1200  
}
```

3. Connection

You have now seen several ways to store data:

```
MySQL  
→ relational tables  
→ rows and columns  
  
MongoDB  
→ collections  
→ documents  
  
Redis  
→ keys and values
```

MySQL might store:

id	name	price
1	Keyboard	1200

MongoDB might store:

```
{  
    "name": "Keyboard",  
    "price": 1200  
}
```

```
}
```

Redis can store a simple value using a key:

```
product:1:price → 1200
```

Redis is especially useful when an application needs to retrieve certain data **very quickly**.

4. Important Topics

Redis

Redis is a fast key-value data store.

At the beginner level, imagine it as a large collection of:

```
key → value
```

For example:

```
username:1 → Asha  
product:1:price → 1200  
city:1 → Bengaluru
```

Your application supplies a key, and Redis can return its associated value.

Key

A **key** is the name used to identify some stored data.

Example:

```
product:101:price
```

Keys should usually describe what the value represents.

For example:

```
Bad:  
price  
  
Better:  
product:101:price
```

The second key makes it clearer which product the price belongs to.

Value

A **value** is the data stored under a key.

For example:

```
Key:
product:101:price

Value:
1200
```

So conceptually:

```
product:101:price → 1200
```

SET

SET stores a value under a key.

Conceptually:

```
SET product:101:price 1200
```

Think:

| Store 1200 using the key product:101:price.

Using a Python Redis client, the idea is similar to:

```
redis_client.set("product:101:price", 1200)
```

GET

GET retrieves the value stored under a key.

Conceptually:

```
GET product:101:price
```

Redis might return:

```
1200
```

In Python, the idea is:

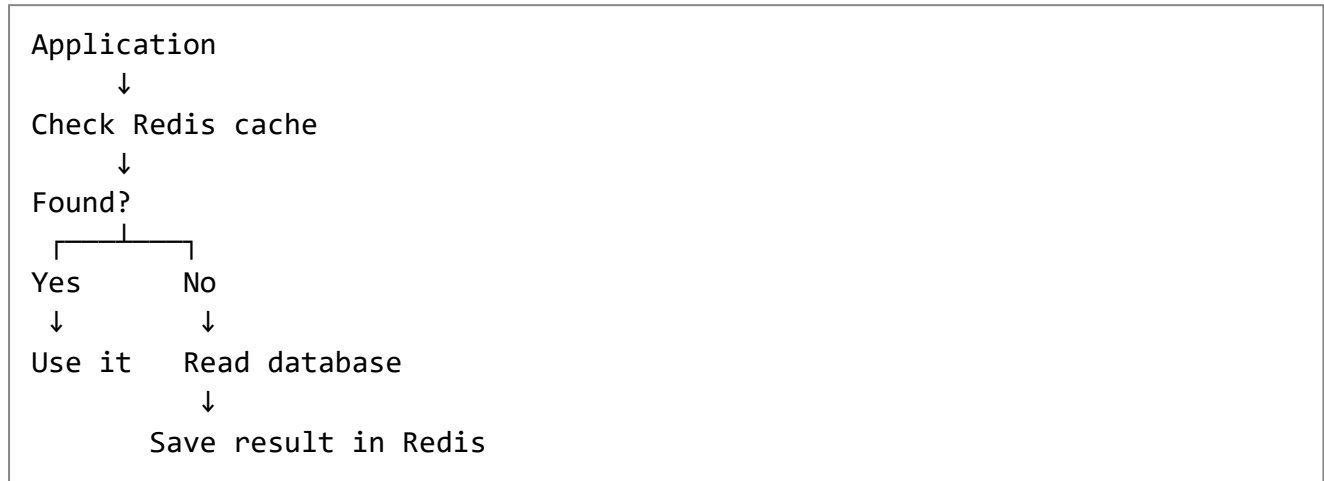
```
redis_client.get("product:101:price")
```

Cache Concept

One very common Redis use is **caching**.

A cache stores frequently needed data somewhere that is quick to access.

For example:



This can prevent the application from repeatedly asking the main database for the same information.

Expiration Concept

Redis can also store a value temporarily.

For example:

```
product:101:price → 1200
```

might be stored for:

```
60 seconds
```

After that time, Redis can automatically remove the key.

This is called **expiration** or a **time-to-live (TTL)**.

You can think of it as:

```
Store this value,  
but only keep it temporarily.
```

This is very useful for caches because cached information may become outdated.

5. Foundational Notes

Redis usually runs as a separate service, similar to a database server.

Your Python program communicates with it through a Redis client library.

Conceptually:

```
Python
  ↓
Redis client
  ↓
Redis server
  ↓
stored key/value
```

A popular Python package is:

```
redis
```

For local practice, it can commonly be installed with:

```
pip install redis
```

You would then conceptually start with:

```
import redis
```

At this stage, focus on the basic flow rather than Redis server configuration.

6. Redis vs MySQL and MongoDB

The three systems organize data differently.

System	Basic storage idea	Example
MySQL	Tables, rows, columns	Product row
MongoDB	Collections and documents	Product document
Redis	Keys and values	Product price key

Imagine a product:

```
Keyboard
Price = 1200
Stock = 10
```

MySQL

```
id | name      | price | stock
1  | Keyboard  | 1200  | 10
```

MongoDB

```
{
  "name": "Keyboard",
  "price": 1200,
  "stock": 10
}
```

Redis

For a very simple value:

```
product:1:price → 1200
```

Redis is often used alongside another database rather than automatically replacing it.

For example:

```
MySQL
→ permanent product records

Redis
→ temporarily cached product prices
```

7. What Is a Cache?

Suppose your application repeatedly needs the price of a popular product.

Without a cache:

```
Request 1 → MySQL
Request 2 → MySQL
Request 3 → MySQL
Request 4 → MySQL
```

The database keeps answering the same question.

With Redis as a cache:

```
First request
  ↓
Redis has no value
  ↓
Read MySQL
  ↓
Price = 1200
  ↓
Save 1200 in Redis
```

Later requests can become:

```
Request
  ↓
Redis
  ↓
1200
```

So the main database may not need to perform the same lookup every time.

A cache is therefore:

A temporary place for frequently needed data so it can often be retrieved more quickly.

The main database can remain the primary source of the permanent data.

8. Easy Example

Suppose you want to store the price of a keyboard.

Choose this key:

```
product:keyboard:price
```

and this value:

```
1200
```

Conceptually:

```
SET product:keyboard:price 1200
```

Then retrieve it:

```
GET product:keyboard:price
```

Expected Redis result:

```
1200
```

The equivalent Python idea is:

```
client.set("product:keyboard:price", 1200)

price = client.get("product:keyboard:price")
```

Depending on the Python Redis client's configuration, returned Redis values may appear as bytes, such as:

```
b'1200'
```

You can handle conversion later; for today, focus on `SET` and `GET`.

9. Problem Statement

Imagine you have a product:

```
Product: Mouse  
Price: 500
```

Create a very small Redis exercise that:

1. connects conceptually to Redis
2. creates a key representing the mouse's price
3. stores `500` using that key
4. reads the value back
5. prints the returned price
6. explains how the same key could act as a cache for a price normally stored in MySQL

For example, your key could follow a pattern like:

```
product:mouse:price
```

Do not build a large application.

10. Concepts Used

You will use:

- Redis
- key-value storage
- key
- value
- `SET`
- `GET`
- Python Redis client
- cache
- cache hit idea
- cache miss idea
- expiration
- database lookup concept

A useful pair of terms is:

Cache hit
→ Redis already contains the value

Cache miss
→ Redis does not contain the value

For now, understanding the idea is enough.

11. Thought Process

First ask:

What value do I want to store?

The product price:

500

How will Redis identify it?

Create a descriptive key:

product:mouse:price

Now you have:

product:mouse:price → 500

How do I store it?

Think:

SET

How do I retrieve it?

Think:

GET

What if this is being used as a cache?

Your application could first ask Redis:

Does product:mouse:price exist?

If Redis returns the value:

```
use it
```

If Redis does not return the value:

```
read the price from MySQL
↓
store it in Redis
↓
use the price
```

12. Beginner-Friendly Pseudocode

```
START

connect to Redis

create key:
    "product:mouse:price"

store:
    key → 500

read value using key

IF value exists:
    print price
ELSE:
    print "Price not found"

END
```

For the caching idea:

```
START

look for product price in Redis

IF price exists:
    use cached price

ELSE:
    read price from main database

    save price in Redis
```

```
use database price
```

```
END
```

The important cache sequence is:

```
Check Redis
  ↓
Found? — Yes → use cached value
      |
      No
      ↓
Read database
  ↓
Save in Redis
  ↓
Use value
```

13. Easy Edge Cases

Key Does Not Exist

Suppose you ask Redis for:

```
product:monitor:price
```

but that key was never stored.

Conceptually:

```
GET product:monitor:price
```

will not return a product price.

Your Python logic should be prepared for:

```
value exists
```

or:

```
value does not exist
```

For example:

```
IF price was found:
    print price
ELSE:
```

```
print "Price not found"
```

If Redis is being used as a cache, a missing key could tell the application:

```
Go to the main database and get the value.
```

Expired Key

Suppose the cache contains:

```
product:mouse:price → 500
```

and it was configured to expire after a certain amount of time.

Once that time passes:

```
product:mouse:price
```

may no longer exist.

The next lookup becomes a cache miss.

Conceptually:

```
Redis key expired
↓
GET returns no cached price
↓
Read current price from database
↓
Cache it again
```

This helps prevent old cached values from remaining forever.

14. Expected Result

Suppose you store:

```
Key:
product:mouse:price

Value:
500
```

After `SET`, Redis conceptually contains:

```
product:mouse:price → 500
```

Then:

```
GET product:mouse:price
```

should return:

```
500
```

Your Python output might conceptually be:

```
Product price: 500
```

For caching, the expected behavior is:

```
First request:
Redis → not found
MySQL → 500
Redis stores → 500

Later request:
Redis → 500
No repeated database lookup needed
```

15. Hint Only

Start with just one key and one value:

```
product:mouse:price → 500
```

Ask yourself:

```
Which Redis operation stores a value?
→ SET

Which Redis operation retrieves it?
→ GET
```

For the caching part, remember:

```
check Redis first

IF value exists:
    use it
```

ELSE:

get it from the database
save it in Redis

Do not worry about clusters, streams, pub/sub, or advanced Redis structures yet.

The key idea for **Day 88** is:

Redis stores data using keys and values. `SET` stores a value, `GET` retrieves it, and Redis can be used as a cache so an application does not need to repeatedly fetch the same data from its main database.